Abdullah Alnamlah, Joseph Hauerstein,
Emma Perez, Mahek Shah,
Anderson Romero

Submission 2

1 Flow Chart

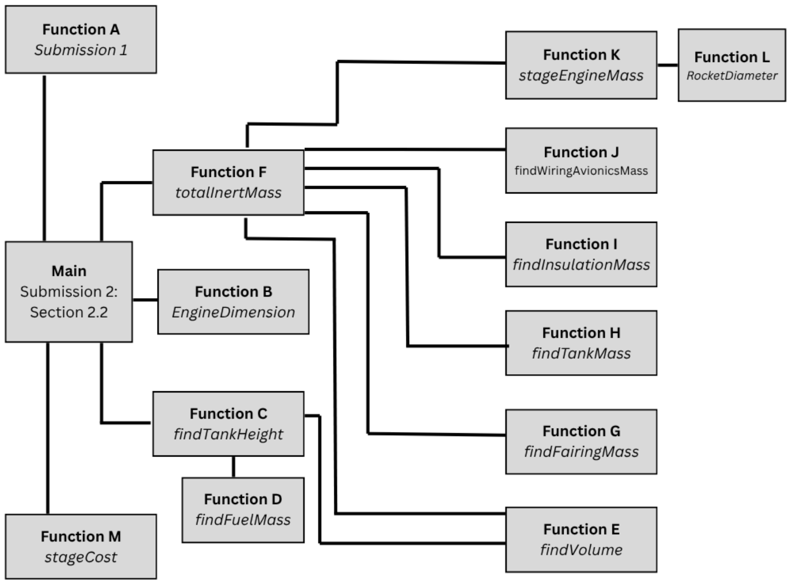

Figure 1: Flow Chart of Function Organization

2 Code

```
%% Main Section Code for 2.1
% All team members worked on this code

clear; clc; close all;
%propNames = ["LOX/LCH4" "LOX/LH2" "LOX/RP1" "Solid" "Storables"];
propNames = ["LOX/LH2"];
Propellantstage1 = "LOX/LCH4"; % user changed

h = 4; % meters, we decided as a team vote
delta = 0.08;
%tolarence 0.05 for mass margin

vehicleParamsSize = 7; % if you add a vehicleParam later, change this number
allOptimizedVehicles = zeros(length(propNames)*2,vehicleParamsSize);
massMargins = zeros(length(propNames),1);
totalMasses = zeros(2,length(propNames));
chiValues = zeros(length(propNames),1);
deltaValues = zeros(length(propNames),1);

for r=0:1

for k = 1:length(propNames)     % Going through all the propellant names/combinations

    firstIteration = true;
    mass_margin = 0;

    while mass_margin < 0.3 || mass_margin > 0.31    % This is to update the delta, if it is not the first iteration, until a mass margin of 30% is achieved

        if ~ firstIteration
            if mass_margin < 0.3
                %delta = delta + 0.0001;
                % adaptive step
                delta = delta + max(abs(mass_margin-0.3)/100, 1e-5);
            elseif mass_margin > 0.301
                %delta = delta - 0.0001;
                % adaptive step
                delta = delta - max(abs(mass_margin-0.3)/100, 1e-5);
            end
        else
            firstIteration = false;
        end

        Propellants = [Propellantstage1, propNames(k)];     % user's specific propellant combination
        [ Mo_min, Min1_min,Min2_min,Mo1_min, Mo2_min,Mpr1_min,Mpr2_min, chi_min] = submission1 (delta, Propellantstage1, propNames(k), r); % Grab submission 1 masses
        chiValues(k) = chi_min;

```

```
        vehicle_inertMass1 = Min1_min + Min2_min;
       m0 = [Mo_min, Mo2_min];      % stage 1 and 2 array
        totalMasses(:,k) = m0;
       Mpr0 = [Mpr1_min Mpr2_min]; % stage 1 and 2 array
        thrust_weight_ratio = [1.3 0.76];
        vehicle_inertMass2 = 0;
        previous_radius = 0;
        previous_stage = [];

        vehicleParams = [[]; []];

        for i=2:-1:1    % start with stage 2 and go to stage 1
            stage = i;
            [nEngines, diameter_ofthrust] = EngineDimension(stage, m0(stage), Propellants(stage));  % find number of engines and diameter of all those engines
            radius = diameter_ofthrust/2;

            [height, fuelh, h_oxidizer, ratio, rho] = findTankHeight(Mpr0(i), radius, Propellants(stage));     % function to find height of tank
            min_radius = Inf;   % intializing variable, high number
            min_height = Inf;
            minInertmass = Inf; % intializing variable, high number
            minstageArray = [];
            minEngines = Inf;
            lastHeight = Inf;

            firstRun = true;

            while (radius < height*10 && height ~= lastHeight) || firstRun % constraint
                firstRun = false;
                radius = radius + 0.1; % keep increasing radius until constraint is met

                lastHeight = height;
                [height, fuelh, h_oxidizer, ratio, rho] = findTankHeight(Mpr0(i),radius, Propellants(i));  % new height based on new radius
                L = height;
                D = radius*2;

                if L/D > 13 || radius < previous_radius || D < 5.3 % if condition not met then skip to the next iteration
                    continue
                end

                %array = [1:stage height, 2:radius of stage, 3:height of fuel tank,
                % 4:height of oxidizer tank, 5:Propellant mass, 6:fuel fraction,
                % 7:oxidizer fraction, 8:fuel density, 9:oxidizer density, 10:initial mass
```

```
                % 11: Thrust to Weight ratio, 12:number of engines]
                stageArray = [height, radius, fuelh, h_oxidizer, Mpr0(stage), ratio(2), ratio(1), rho(2), rho(1), m0(stage), thrust_weight_ratio(stage), nEngines];
                stage1_array = [];
                stage2_array = stageArray;
                if stage == 1
                    stage1_array = stageArray;
                    stage2_array = previous_stage;
                end

            [InertMass, stage, propellant_names] = totalInertMass(stage1_array, stage2_array, Propellants(stage), m0(stage), h, stage);
            currentInertMass = sum(InertMass);

                if currentInertMass < minInertmass   % check for new minimum inert mass
                    min_radius = radius;                    % new minimum radius
                    min_height = height;                    % new minimum height
                    minInertmass = currentInertMass;        % new minimum inert mass
                    minstageArray = stageArray;
                    minEngines = nEngines;
                end
            end
            previous_radius = min_radius;       % update the previous radius
            vehicle_inertMass2 = minInertmass + vehicle_inertMass2;      % add the inert masses
            previous_stage = minstageArray;

            vehicleParams(stage,:) = [minEngines min_radius min_height minInertmass Mpr0(stage) m0(stage) thrust_weight_ratio(stage)];

        end

            allOptimizedVehicles(2*k-1:2*k, :) = vehicleParams;
            mass_margin = (vehicle_inertMass1 - vehicle_inertMass2)/(vehicle_inertMass2);  % calculate the mass margin
            massMargins(k) = mass_margin
            deltaValues(k) = delta;

    end % end of while loop

end

for k=1:(length(allOptimizedVehicles(:,1))/2)
    disp(Propellantstage1 + ", " + propNames(k))
    for l=0:1
        index = 2*k+l-1;
    fprintf("Stage: %d \tEngines: %d \tRadius: %f \tHeight: %f \tInert Mass: %d \t\nPropellant Mass: %d \tInitial Stage Mass: %d \tTWR: %d \tMass Margin: %d\n\n", ...
        l+1, ...
```

```
            allOptimizedVehicles(index, 1), ...
            allOptimizedVehicles(index, 2), ...
            allOptimizedVehicles(index, 3), ...
            allOptimizedVehicles(index, 4), ...
            allOptimizedVehicles(index, 5), ...
            allOptimizedVehicles(index, 6), ...
            allOptimizedVehicles(index, 7), ...
            massMargins(k))
    end
end

for k=1:(length(allOptimizedVehicles(:,1))/2)
    index = 2*k-1;
    nEngines = [allOptimizedVehicles(index, 1) allOptimizedVehicles(index+1, 1)];
    radii = [allOptimizedVehicles(index, 2) allOptimizedVehicles(index+1, 2)];
    heights = [allOptimizedVehicles(index, 3) allOptimizedVehicles(index+1, 3)];

    Mpr0 = [allOptimizedVehicles(index, 5) allOptimizedVehicles(index+1, 5)];
    [height1, fuelh1, h_oxidizer1, ratio1, rho1] = findTankHeight(Mpr0(1), radii(1), Propellantstage1) % new height based on new radius
    [height2, fuelh2, h_oxidizer2, ratio2, rho2] = findTankHeight(Mpr0(2), radii(2), propNames(k)) % new height based on new radius]
    fuel_heights = [fuelh1 fuelh2];
    oxidizer_heights = [h_oxidizer1 h_oxidizer2];
    ratios = [ratio1; ratio2];
    rhos = [rho1; rho2];
    thrust_weight_ratio = [1.3 0.76];
    m0 = totalMasses(:,k);

    stageArray1 = [heights(1), radii(1), fuel_heights(1), oxidizer_heights(1), Mpr0(1), ratios(2,1), ratios(1,1), rhos(2,1), rhos(1,1), m0(1), thrust_weight_ratio(1), nEngines(1)];
    stageArray2 = [heights(2), radii(2), fuel_heights(2), oxidizer_heights(2), Mpr0(2), ratios(2,2), ratios(1,2), rhos(2,2), rhos(1,2), m0(2), thrust_weight_ratio(2), nEngines(2)];

    % Need: Stage, Chi, Propellant,
    % Propellant Tanks S1, Propellant Tanks S2, Tank Insulation S1, Tank Insulation S2,
    % Engine S1, Engine S2, Thrust Structure S1, Thrust Structure S2, Casing S1, Casing S2, Gimbals S1, Gimbals S2,
    % Avionics (S2), Wiring S1, Wiring S2,
    % Payload Fairing, Inter Tank Fairing S1, Inter Tank Fairing S2, Inter Stage Fairing, Aft Fairing

    S2Avionics = 0;
    inertMassSum = 0;
    costSum = 0;

    Propellants = [Propellantstage1, propNames(k)];

    fprintf("Propellants: %s %s\n\n", Propellants(1), Propellants(2))
```

```
for stage=1:2

    [InertMass, ~, propellant_names] = totalInertMass(stage1_array,
        stage2_array,Propellants(stage),m0(stage),h,stage);

    % Array guide for subsystem masses:
    % 1 --> if i = 1 , propellant_name = "Solid" : totalMass1 = [Interstage
        Aft1 propellantTank insul_solid engineMass1 structureMass1
        gimbalsMass1 wiringMass1 avionicsMass1];
    % 2 --> if i = 1 , propellant_name = "else" : totalMass1 = [Intertank1
        Interstage Aft1 fuel_tank1 oxidizer_tank1 insul_oxid1 insul_fuel1
        engineMass1 structureMass1 gimbalsMass1 wiringMass1 avionicsMass1 ];
    % 3 --> if i = 2 , propellant_name = "Solid" : totalMass2 = [Payload2
        propellantTank2 insul_solid2 engineMass2 structureMass2 gimbalsMass2
        wiringMass2];
    % 4 --> if i = 2 , propellant_name = "else" : totalMass2 = [payload2
        Intertank2 fuel_tank2 oxidizer_tank2 insul_oxid2 insul_fuel2
        engineMass2 structureMass2 gimbalsMass2 wiringMass2];

    displayString = "Stage: %d\tChi: %f\tPropellant: %d\tDelta: %f\n" + ...
        "Propellant Tanks: %d\tTank Insulation: %d\t\n" + ...
        "Engines: %d\tThrust Structure: %d\tCasing: %d\tGimbals: %d\t\n" + ...
        "Avionics: %d\tWiring: %d\n" + ...
        "Payload Fairing: %d\tIntertank Fairing: %d\tInterstage Fairing: %d\
            tAft Fairing: %d\n\n";
    if (stage == 1 && strcmp(propellant_names, "Solid"))
        S2Avionics = InertMass(9);
        fprintf(displayString, ...
            stage, chiValues(k), Mpr0(stage), deltaValues(k), ...
            InertMass(3), InertMass(4), ...
            0, InertMass(6), InertMass(5), InertMass(7), ...
            0, InertMass(8), ...
            0, 0, InertMass(1), InertMass(2));
    elseif (stage == 1 && strcmp(propellant_names, "else"))
        S2Avionics = InertMass(12);
        fprintf(displayString, ...
            stage, chiValues(k), Mpr0(stage), deltaValues(k), ...
            InertMass(4)+InertMass(5), InertMass(6)+InertMass(7), ...
            InertMass(8), InertMass(9), 0, InertMass(10), ...
            0, InertMass(11), ...
            0, InertMass(1), InertMass(2), InertMass(3));
    elseif (stage == 2 && strcmp(propellant_names, "Solid"))
        fprintf(displayString, ...
            stage, chiValues(k), Mpr0(stage), deltaValues(k), ...
            InertMass(2), InertMass(3), ...
            0, InertMass(5), InertMass(4), InertMass(6), ...
            S2Avionics, InertMass(7), ...
            InertMass(1), 0, 0, 0);
    elseif (stage == 2 && strcmp(propellant_names, "else"))
        fprintf(displayString, ...
            stage, chiValues(k), Mpr0(stage), deltaValues(k), ...
            InertMass(3)+InertMass(4), InertMass(5)+InertMass(6), ...
            InertMass(7), InertMass(8), 0, InertMass(9), ...
```

```
            S2Avionics, InertMass(10), ...
            InertMass(1), InertMass(2), 0, 0);
    end
    cost = stageCost(sum(InertMass));
    fprintf("Stage Cost: %d\n\n", cost);
    costSum = costSum + cost;
end
    fprintf("Vehicle Cost: %d\n\n", costSum);
end

end
```

```
function [nEngines, diameter_ofthrust] = EngineDimension(stage,m0,prop) % Emma
    and Joseph mainly worked on code
    propNames = ["LOX/LCH4" "LOX/LH2" "LOX/RP1" "Solid" "Storables"]; %
        propellant names
    i = find(propNames==prop) ;
    g = 9.81;
    TWR = [1.3 0.76]; % Thrust to weight ratio
    Treq = [2.26 1.86 1.92 4.5 1.75; 0.745 0.99 0.61 2.94 0.67]; % Thrusts
    nEngines = (Treq(stage,i)*10^6/(m0*g*TWR(stage)))^-1;
    nEngines = ceil(nEngines); % Round engine count up to whole integer
    D_engine = [2.4 2.4 3.7 3.3 1.5; 1.5 2.15 0.92 2.34 1.13]; % Engine
        diamaters
    diameter = D_engine(stage, i);
    [radius_ofthrust] = RocketDiameter (nEngines,diameter);
    diameter_ofthrust = radius_ofthrust*2;
end
```

```
function [Payload, Intertank1 ,Intertank2 , Interstage, Aft] = findFairingMass
    (rs1, rs2,h)
%% Call this function twice for each stage
% Abdullah mainly worked on this code
%rs1: Radius of 1st stage
%rs2: Radius of 2nd stage
%h: height of aerodynamic cone

A_cone = pi*rs2*sqrt(rs2^2 + h^2); %Area for the aerodynamic fairing
A_cylinder_payload = 2*pi*rs2*(13); %Area for payload cylinder
A_payload = A_cone + A_cylinder_payload; %Total area of payload fairing
A_frustrum = pi*(rs1 + rs2)*sqrt((rs1-rs2)^2+(rs1+rs2+3)^2);%Area for inter-
    stage fairing

% leave sufficient engine fairing below propellant tanks (3m)
A_cylinder1 = 2*pi*rs1*(2*rs1); %Area for inter-tank fairing, stage 1
A_cylinder2 = 2*pi*rs2*(2*rs2); %Area for inter-tank fairing, stage 2
A_aft = 2*pi*rs1*(rs1+3); %Area for aft fairing

%caluclate fairing masses
Payload = 4.95*(A_payload)^1.15;%Only for 2nd stage
Intertank1 = 4.95*(A_cylinder1)^1.15; %Inter-tank fairng for 1st stage
Intertank2 = 4.95*(A_cylinder2)^1.15; %Inter-tank fairng 2nd stage
Interstage = 4.95*(A_frustrum)^1.15; %One inter-stage fairing
Aft = 4.95*(A_aft)^1.15;%Aft fairing of 2nd stage = Inter-stage fairing
```

```
end
```

```
%% Oxidizer/fuel Mass finder
% Abdullah mainly worked on this code
function [oxidizer_mass, fuel_mass] = findFuelMass(Mpr0, ratio_array)
    %Mpr0 = total propellant mass per stage
    %ratio_array(1) = fuel fraction
    %ratio_array(2) = oxidizer fraction
    tot_ratio = ratio_array(1) + ratio_array(2);
    oxz_ratio = ratio_array(1)/tot_ratio;
    ful_ratio = ratio_array(2)/tot_ratio;
    oxidizer_mass = oxz_ratio*Mpr0;
    fuel_mass = ful_ratio*Mpr0;
end
```

```
%% Find Tank Insulation
% Anderson and Mahek mainly worked on this code
function InsMass = findInsulationMass(AT,fuelname) %exposed surface area of
    tank, fuel name
    if strcmp(fuelname, "LH2")
        InsMass = 2.88*AT;
    elseif strcmp(fuelname, "LCH4") || strcmp(fuelname, "LOX")
        InsMass = 1.123*AT;
    else
        InsMass = 0;
    end
end
```

```
function [h_total, fuel_height, oxidizer_height, ratio, rho, r] =
    findTankHeight(Mpr0,r, Propellant)
% Abdullah Mahek Anderson mainly worked on this code
%Mpr0: Overall Propellant mass (kg)
%r:
    % Find height of the tank cylinder based on propellant mass and radius
density = [71 1140 820 423 1680 1442 791]; % LH2 LOX RP-1 LCH4 APCP(solid)
    N2O4 UDMH
% NOTE: propMass must include oxidizer first
if Propellant == "LOX/LCH4"
    rho = [density(2) density(4)];
    ratio = [3.6 1];
    [oxidizer_mass, fuel_mass] = findFuelMass(Mpr0, ratio);
    propMass = [oxidizer_mass, fuel_mass];
elseif Propellant == "LOX/LH2"
    rho = [density(2) density(1)];
    ratio = [6.03 1];
    [oxidizer_mass, fuel_mass] = findFuelMass(Mpr0, ratio);
    propMass = [oxidizer_mass, fuel_mass];
elseif Propellant == "LOX/RP1"
    rho = [density(2) density(3)];
    ratio = [2.72 1];
    [oxidizer_mass, fuel_mass] = findFuelMass(Mpr0, ratio);
    propMass = [oxidizer_mass, fuel_mass];
elseif Propellant == "Solid"
    ratio = [1 0];
```

```
    rho = [density(5) 0];
    propMass = Mpr0;
elseif Propellant == "Storables"
    rho = [density(6) density(7)];
    ratio = [2.67 1];
    [oxidizer_mass, fuel_mass] = findFuelMass(Mpr0, ratio);
    propMass = [oxidizer_mass, fuel_mass];
end

h_total = 0;
heights = [];
    for i = 1:length(propMass)
        volume = findVolume(propMass(i),rho(i));
        %volume for cylinder with hemispherical caps
        %Replace formula below
        if Propellant == "Solid"
            h = volume/(pi*r^2);
            r = 0;
        else
            h = max((volume-4*(pi/3)*r^3) / (pi*r^2), 0);
            if h == 0
                r = (3*volume/(4*pi))^(1/3);
            end
        end

        heights(end+1) = h+2*r;
        h_total = h+h_total+2*r;
    end
    if Propellant == "Solid"
        ratio = [0 1];
        rho = [0 density(5)];
    end
    if (length(heights) == 2)
        oxidizer_height = heights(1);
        fuel_height = heights(2);
    elseif (length(heights) == 1)
        fuel_height = heights(1);
        oxidizer_height = 0;
    end
end
```

```
%% FIND Mass of the Tank
% Abdullah and Joseph mainly worked on this code
function M_tank = findTankMass(Mpr,fuelname,Vpr)
    if strcmp(fuelname, "LOX") %LOX
        M_tank = 0.0107*Mpr;
    elseif strcmp(fuelname, "LH2") %LH2
        M_tank = 0.128*Mpr;
    elseif strcmp(fuelname, "RP1") %RP-1
        M_tank = 0.0148*Mpr;
    elseif strcmp(fuelname, "LCH4") %LCH4
        M_tank = 0.0287*Mpr;
    else %N2O4, UDMH, Solid
        M_tank = 12.16*Vpr;
```

```
    end
end
```

```
%% Volume finder
% Mahek worked on this code
function volume = findVolume(m,rho)
    volume = m/rho;
end
```

```
%% Find wiring and avionics masses
% Abdullah and Anderson mainly worked on this code
function [wirMass,AvcMass] = findWiringa_AvionicsMass(M0,l)
    AvcMass = 10*(M0^0.361);
    wirMass = 1.058*sqrt(M0)*(l^0.25);
end
```

```
function [radius_ofthrust] = RocketDiameter(nEngines,diameter)
% Emma and Joseph mainly worked on this code
radius = diameter/2;
% curve fit of first 20 known solutions for packing a circle in a circle
a = 1.486;
b = 0.4277;
radius_multiplier = a*nEngines^b;
radius_ofthrust = radius_multiplier*radius;
end
```

```
function [engineMass, structureMass, gimbalsMass] = stageEngineMass(m0, mProp,
    TWR, type, nEngine, stageNum)
% Joseph mainly worked on this code
    %m0: total mass per stage (kg)
    %mprop: propellant mass per stage (kg)
    %TWR: thrust to weight ratio
    %Type: propellant name (fuel/oxidizer)
    %nEngine: number of engines
    %stageNum: stage number
    g = 9.81;
    propNames = {'LOX/LCH4','LOX/LH2','LOX/RP1','Solid','Storables'};
    % arrays of thrust, chamber pressure, and expansion ratios per stage per
    % propellant
    s1Thrust = [2.26e+6, 1.86e+6, 1.92e+6, 4.5e+6, 1.75e+6];
    s2Thrust = [0.745e+6, 0.099e+6, 0.061e+6, 2.94e+6, 0.067e+6];
    s1Pressure = [35.16e+6, 20.64e+6, 25.8e+6, 10.5e+6, 15.7e+6];
    s2Pressure = [10.1e+6, 4.2e+6, 6.77e+6, 5e+6, 14.7e+6];
    s1ExpantionRatio = [34.34, 78, 37, 16, 26.2];
    s2ExpantionRatio = [45, 84, 14.5, 56, 81.3];
    %Loop through propellant names and find values based on propellant pick
    for i = 1:length(propNames)
        if strcmp(type, propNames(i))
            if stageNum == 1
                TPerEngine = s1Thrust(i);
                P0 = s1Pressure(i);
                expanRatio = s1ExpantionRatio(i);
            end
            if stageNum == 2
```

```
                TPerEngine = s2Thrust(2);
                P0 = s2Pressure(i);
                expanRatio = s2ExpantionRatio(i);
            end
        end
    end
    T_total = TWR * m0 * g;%Total thrust (N)
    structureMass = 2.55*10^-4*T_total;%overall structure mass
    gimbalsMass = nEngine*(237.8*(TPerEngine/P0)^0.9375);%Gimbal mass per engine

    if strcmp(type, "LOX/LH2") || strcmp(type, "LOX/LCH4") || strcmp(type, "LOX/RP1") || strcmp(type, "Storables")
        engineMass = 7.81*10^-4*TPerEngine + 3.37*10^-5*TPerEngine*sqrt(expanRatio) + 59;% Engine mass
        engineMass = nEngine*engineMass;
    end
    if strcmp(type, "Solid")
        engineMass = 0.135*mProp;
    end

end
```

```
function [ Mo_min, Min1_min,Min2_min,Mo1_min, Mo2_min,Mpr1_min,Mpr2_min,chi_min] = submission1 (delta, Propellantstage1, Propellantstage2, useMinMass)
% Joseph and Emma mainly worked on this code
% Givens
delta_v = 12.3; % km/s
m_pl = 26000; % kg
chi = 0.2:0.01:0.8; % array
Isp = [327,    366,    311,    269,    285];
propNames = ["LOX/LCH4" "LOX/LH2" "LOX/RP1" "Solid" "Storables"];
for i = 1:length(propNames) %run through all propellant names
     %select Isp based on prop. name and stage
    if strcmp(Propellantstage1, propNames(i))
        Isp1 = Isp(i);
    end
     if strcmp(Propellantstage2, propNames(i))
        Isp2 = Isp(i);
    end
end

    [M01_array, M02_array, chi_array] = getMass(delta_v,m_pl,delta,chi,Isp1,Isp2); %getting stage mass for all chi values
    [m_pr1, m_pr2] = propMass(delta, M01_array, M02_array, m_pl); %propellant mass for both stages
    [m_in1, m_in2] = inertMass(delta, M01_array, M02_array); %inert mass for both stages
    Mo = M01_array + M02_array; %total initial mass

    if (~useMinMass)
    %calculate stage cost based on intert mass
    costS1 = stageCost(m_in1);
```

```
    costS2 = stageCost(m_in2);
    costTotal = costS1 + costS2;
    [~, cIndex] = findMinCost(costTotal); %finding the minimum of the costs

    %getting all the minimum cost points for the different mass parameters
    Mo_min = Mo(cIndex);
    Min1_min = m_in1(cIndex);
    Min2_min = m_in2(cIndex);
    Mo1_min = M01_array(cIndex);
    Mo2_min = M02_array(cIndex);
    Mpr1_min = m_pr1(cIndex);
    Mpr2_min = m_pr2(cIndex);
    chi_min = chi_array(cIndex);
    else
    [~, mIndex] = findMinMass(Mo);

    %getting all the minimum cost points for the different mass parameters
    Mo_min = Mo(mIndex);
    Min1_min = m_in1(mIndex);
    Min2_min = m_in2(mIndex);
    Mo1_min = M01_array(mIndex);
    Mo2_min = M02_array(mIndex);
    Mpr1_min = m_pr1(mIndex);
    Mpr2_min = m_pr2(mIndex);
    chi_min = chi_array(mIndex);
    end
end
```

```
%% Array of givens
% All team members worked on this code
%array = [1:stage height, 2:radius of stage, 3:height of fuel tank,
% 4:height of oxidizer tank, 5:Propellant mass, 6:fuel fraction,
% 7:oxidizer fraction, 8:fuel density, 9:oxidizer density, 10:initial mass
% 11: Thrust to Weight ratio, 12:number of engines]
%% Seperate solid fuel from other propellants in calculations

%Names = [1:stage 1 fuel name, 2:stage 1 oxidizer name, 3:stage 1 propellant name
%          4:stage 2 fuel name, 5:stage 2 oxidizer name, 6:stage 2 propellant name]

function [InertMass, i, propellant_names]=totalInertMass(stage1,stage2,propellantname,M0,h,i)
  big_names_array = ["LOX", "LCH4", "LOX/LCH4"; "LOX", "LH2", "LOX/LH2"; "LOX", "RP1", "LOX/RP1"; "Solid", "", "Solid"; "Storables", "", "Storables"];
  startingIndex = 1;
    if i == 2
        startingIndex = 4;
    end
  names = ["","","","","",""];
  names(startingIndex : startingIndex+2) = big_names_array(big_names_array(:,3) == propellantname,:);
%% Stage one
```

```
  if i == 1
    if strcmp(names(3),"Solid")
        propellant_names = "Solid";
        %Fairing for solid propellants on first stage is Aft & Interstage fairing
        [~, ~ ,~ , Interstage, Aft1] = findFairingMass(stage1(2),stage2(2),h);
        %Propellant mass = solid fuel
        solidM = stage1(5);
        %DENSITY OF SOLID PROPELLANT SHOULD BE STORED IN FUEL DENSITY
        solidV = findVolume(solidM,stage1(8));
        propellantTank = findTankMass(solidM, names(3),solidV);
        %find Surface area of tanks
        solidA = 2*pi*stage1(2)*stage1(1)+4*pi*stage1(2)^2;
        %find insulation mass
        insul_solid = findInsulationMass(solidA,names(3));
        %find Engine, Casing & Gimbal masses
        [engineMass1, structureMass1, gimbalsMass1] = stageEngineMass(stage1(10), stage1(5), stage1(11), names(3), stage1(12), 1);
        [~,avionicsMass1] = findWiringa_AvionicsMass(M0,stage2(1));
        [wiringMass1,~] = findWiringa_AvionicsMass(M0,stage1(1));
%         totalMass1 = Interstage+Aft1+propellantTank+insul_solid+engineMass1+structureMass1+gimbalsMass1+wiringMass1+avionicsMass1;
        totalMass1 = [Interstage Aft1 0 insul_solid engineMass1 structureMass1 gimbalsMass1 wiringMass1 avionicsMass1];
    else
        propellant_names = "else";
        [~, Intertank1 ,~ , Interstage, Aft1] = findFairingMass(stage1(2),stage2(2),h);
        %Fuel mass & Oxidizer masses
        %Pass propellant mass and ratio
        [Moxidizer1, Mfuel1] = findFuelMass(stage1(5), [stage1(6), stage1(7)]);
        %Find volume of fuel & oxidizer
        V_fuel1 = findVolume(Mfuel1,stage1(8));%Pass fuel density
        V_oxid1 = findVolume(Moxidizer1,stage1(9));%Pass oxidizer density
        %Find tanks masses
        fuel_tank1 = findTankMass(Mfuel1,names(1),V_fuel1);
        oxidizer_tank1 = findTankMass(Moxidizer1,names(2),V_oxid1);
        %find Surface area of tanks
        Asurf_fuel1 = 2*pi*stage1(2)*stage1(3) +4*pi*stage1(2)^2;%surface area of fuel tank
        Asurf_oxidizer1 = 2*pi*stage1(2)*stage1(4) +4*pi*stage1(2)^2;%surface area of oxidizer tank
        insul_fuel1 = findInsulationMass(Asurf_fuel1,names(1)); %insulaiton mass for fuel tank
        insul_oxid1 = findInsulationMass(Asurf_oxidizer1,names(2)); %insulation mass for oxidizer tank
        % find Engine, Casing, & gimbal masses
        [engineMass1, structureMass1, gimbalsMass1] = stageEngineMass(stage1(10), stage1(5), stage1(11), names(3), stage1(12), 1);
        %find Wiring mass
        [~,avionicsMass1] = findWiringa_AvionicsMass(M0,stage2(1));
        [wiringMass1,~] = findWiringa_AvionicsMass(M0,stage1(1));
        totalMass1 = [Intertank1 Interstage Aft1 fuel_tank1 oxidizer_tank1
```

```
            insul_oxid1 insul_fuel1 engineMass1 structureMass1 gimbalsMass1 wiringMass1 avionicsMass1 ];
        end
        InertMass = totalMass1;
    end
        %% Stage two
if i == 2
    if strcmp(names(6),"Solid")
        propellant_names = "Solid";
        %Fairing for solid propellants on second stage is payload fairing
        [Payload2, ~ ,~ , ~, ~] = findFairingMass(0,stage2(2),h);
        %Propellant mass = solid fuel
        solidM2 = stage2(5);
        %DENSITY OF SOLID PROPELLANT SHOULD BE STORED IN FUEL DENSITY
        solidV2 = findVolume(solidM2,stage2(8));
        propellantTank2 = findTankMass(solidM2, names(6),solidV2);
        %find Surface area of tanks
        solidA2 = 2*pi*stage2(2)*stage2(1) +4*pi*stage2(2)^2;
        %find insulation mass
        insul_solid2 = findInsulationMass(solidA2,names(6));
        %find Engine, Casing & Gimbal masses
        [engineMass2, structureMass2, gimbalsMass2] = stageEngineMass(stage2(10), stage2(5), stage2(11), names(6), stage2(12), 2);

        [wiringMass2,~] = findWiringa_AvionicsMass(stage2(10),(stage2(1)+13+h));
        %height = 2nd stage height + payload fairing height
        totalMass2 = [Payload2 0 insul_solid2 engineMass2 structureMass2 gimbalsMass2 wiringMass2];
    else
        propellant_names = "else";
        %Fairing mass
        [payload2, ~, Intertank2, ~, ~] = findFairingMass(0,stage2(2),h);
        %Fuel mass & Oxidizer masses
        %Pass propellant mass and ratio
        [Moxidizer2, Mfuel2] = findFuelMass(stage2(5), [stage2(6), stage2(7)]);
        %Find volume of fuel & oxidizer
        V_fuel2 = findVolume(Mfuel2,stage2(8));%Pass fuel density
        V_oxid2 = findVolume(Moxidizer2,stage2(9));%Pass oxidizer density
        %Find tanks masses
        fuel_tank2 = findTankMass(Mfuel2,names(4),V_fuel2);
        oxidizer_tank2 = findTankMass(Moxidizer2,names(5),V_oxid2);
        %find Surface area of tanks
        Asurf_fuel2 = 2*pi*stage2(2)*stage2(3) +4*pi*stage2(2)^2;%surface area of fuel tank
        Asurf_oxidizer2 = 2*pi*stage2(2)*stage2(4) +4*pi*stage2(2)^2;%surface area of oxidizer tank
        insul_fuel2 = findInsulationMass(Asurf_fuel2,names(4)); %insulation mass for fuel tank
        insul_oxid2 = findInsulationMass(Asurf_oxidizer2,names(5)); % insulation mass for oxidizer tank
        % find Engine, Casing, & gimbal masses
        [engineMass2, structureMass2, gimbalsMass2] = stageEngineMass(stage2
```

```
            (10), stage2(5), stage2(11), names(6), stage2(12), 2);
        %find Wiring mass and avionics
        [wiringMass2,~] = findWiringa_AvionicsMass(stage2(10),(stage2(1)+13+h)
            );
        totalMass2 = [payload2 Intertank2 fuel_tank2 oxidizer_tank2
            insul_oxid2 insul_fuel2 engineMass2 structureMass2 gimbalsMass2
            wiringMass2];

    end
    InertMass = totalMass2;
end

end
```